SYSTEMPROVISIONINGANDMONITORINGLAB(CSDV4110P)

LABORATORY MANUAL

Taught by: Dr. Santanu Ghosh
Assistant Professor
CSO, SCOS, UPES

Name: Pallav Bharti
Course: B Tech CSE
Session: 2023-27
Roll No. 500119384
Semester: 7
Batch: 1

Experiment I — Infrastructure as Code (IaC) Basics

Aim

To introduce the concepts of Infrastructure as Code (IaC), to select an appropriate IaC tool, and to write and execute a simple IaC script that provisions a virtual machine on a cloud platform.

Objectives

- To understand the difference between manual provisioning and provisioning through code.

- To install and configure Terraform and a cloud provider CLI.
- To author a declarative configuration that creates a single virtual machine.
- To execute the complete Terraform lifecycle and interpret the output of each stage.
- To place the configuration under version control and destroy the resource cleanly.

Requirements

Hardware

- Laptop or desktop with a minimum of 8 GB RAM and 20 GB free disk space
- A stable internet connection (provider plugins are downloaded at run time)

Software

- Terraform CLI version 1.6 or later
- AWS CLI version 2 (or Azure CLI if Azure is chosen)
- Git version 2.30 or later
- Visual Studio Code with the HashiCorp Terraform extension
- A terminal — Bash on Linux/macOS or PowerShell on Windows

Accounts and access

- An AWS Free Tier account (or Azure for Students)
- An IAM user with programmatic access and the AmazonEC2FullAccess policy attached
- The access key ID and secret access key for that IAM user

Procedure

Step 1 — Install Terraform and verify the installation

Download the Terraform binary for your operating system, extract it and place it on the system PATH. Confirm the installation before proceeding; an incorrect PATH is the most common cause of failure in this experiment.

Verifying the Terraform installation

```
PS C:\Users\palla\Desktop\exp> terraform --version
Terraform v1.13.1
on windows_386
```

Step 2 — Install and configure the AWS CLI

Install AWS CLI version 2 and configure it with the access key of the IAM user created for this laboratory. The credentials are written to ~/.aws/credentials and are picked up automatically by the Terraform AWS provider.

Configuring and verifying cloud credentials

```
PS C:\Users\palla\Desktop\exp> aws configure
AWS Access Key ID [****************KZ6B]: AKIAWXIIQM6QF6GNRENT
AWS Secret Access Key [****************cv6j]: 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
Default region name [ap-south-1]: ap-south-1
Default output format [json]: json
```

```
PS C:\Users\palla\Desktop\exp> aws sts get-caller-identity
{
    "UserId": "462263707552",
    "Account": "462263707552",
    "Arn": "arn:aws:iam::462263707552:root"
}

PS C:\Users\palla\Desktop\exp> cd pallav_exp1
```

Step 3 — Create the project directory and initialise version control

Create a dedicated directory for the experiment, initialise a Git repository inside it and add a .gitignore file before writing any configuration. Adding .gitignore first guarantees that a state file can never be committed by accident.

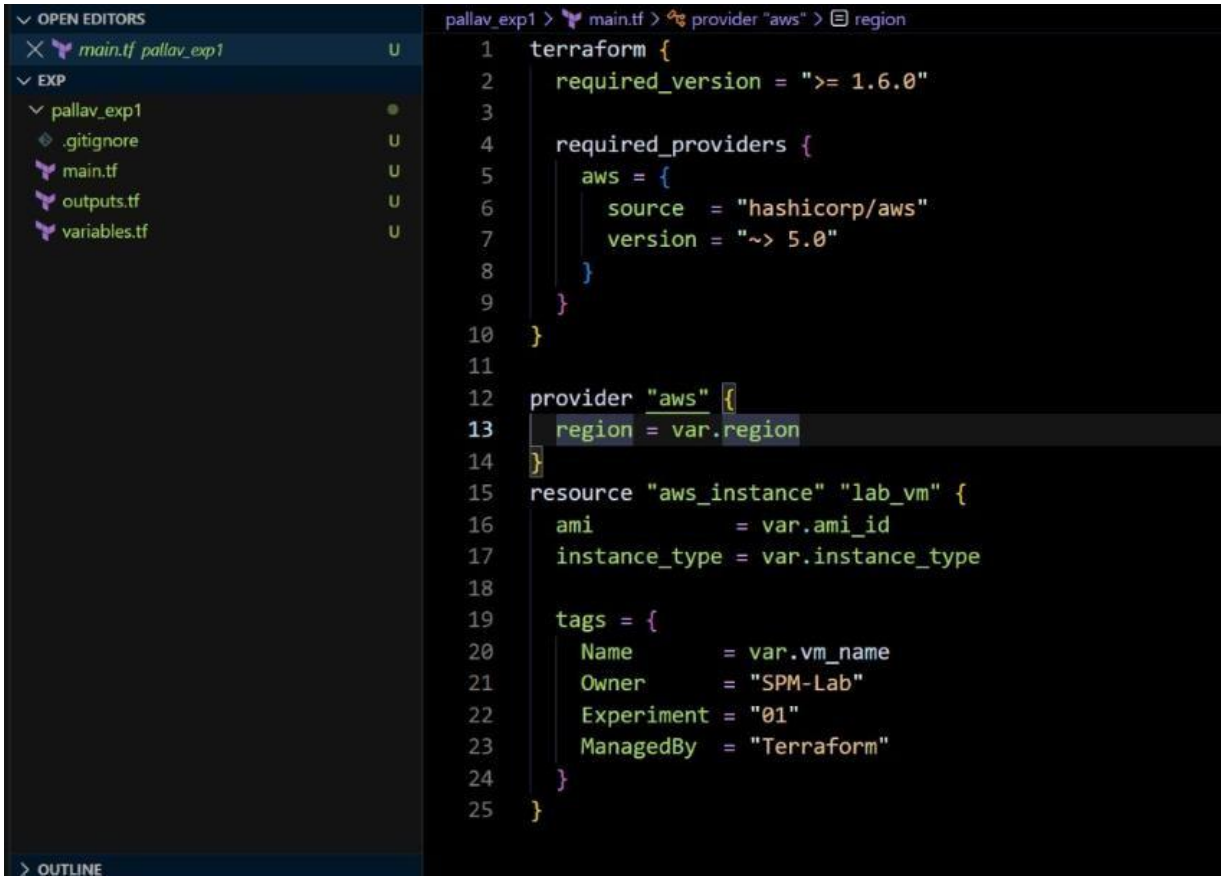
Creating the workspace

```
PS C:\Users\palla\Desktop\exp\pallav_exp1> New-Item .gitignore -ItemType File


    Directory: C:\Users\palla\Desktop\exp\pallav_exp1


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        12-08-2026  09.17 PM              0 .gitignore

PS C:\Users\palla\Desktop\exp\pallav_exp1> Set-Content .gitignore ".terraform/"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Add-Content .gitignore "terraform.tfstate"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Add-Content .gitignore "terraform.tfstate.*"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Add-Content .gitignore "*.tfvars"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Add-Content .gitignore "*.tfvars.json"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Add-Content .gitignore ".terraform.lock.hcl"
PS C:\Users\palla\Desktop\exp\pallav_exp1> git status --short
?? .gitignore
```

```
PS C:\Users\palla\Desktop\exp\pallav_exp1> Set-Content .gitignore ".terraform/"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Add-Content .gitignore "terraform.tfstate"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Add-Content .gitignore "terraform.tfstate.*"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Add-Content .gitignore "*.tfvars"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Add-Content .gitignore "*.tfvars.json"
PS C:\Users\palla\Desktop\exp\pallav_exp1> Get-Content .gitignore
.terraform/
terraform.tfstate
terraform.tfstate.*
*.tfvars
*.tfvars.json
PS C:\Users\palla\Desktop\exp\pallav_exp1>
```

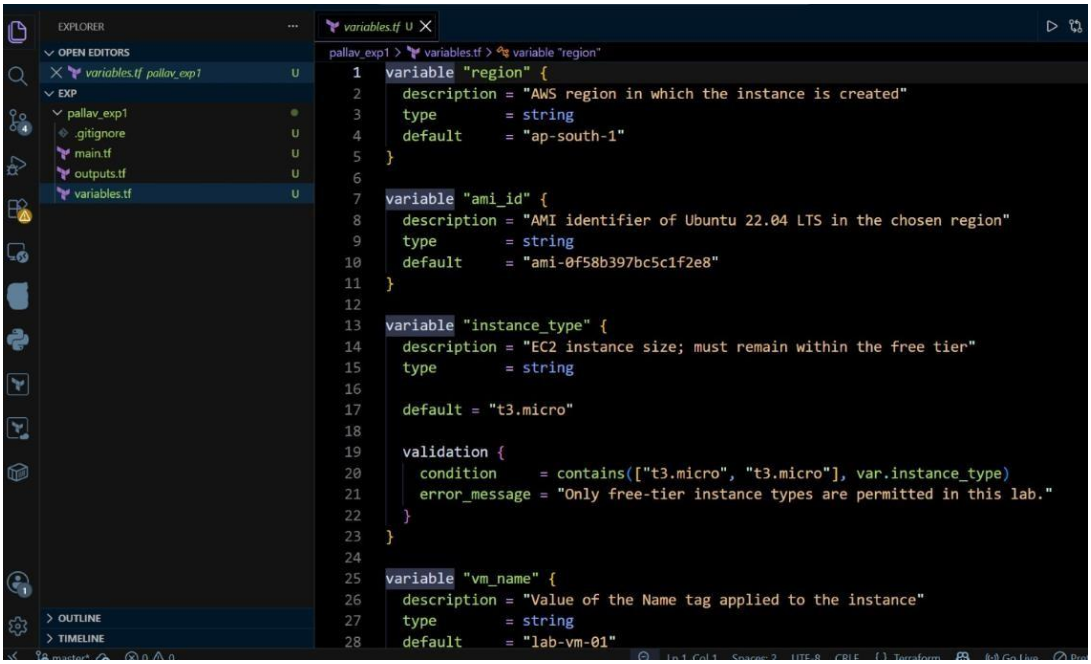
Step 4 — Write the provider configuration

Create main.tf and declare the terraform block with the required provider and its version constraint, followed by the provider block. The constraint "~> 5.0" permits any 5.x version but not 6.0, which prevents a breaking provider release from silently changing the behaviour of the configuration.



Step 5 — Declare input variables

Create variables.tf and declare the region, instance type and machine name as input variables with sensible defaults. Parameterising these values is what makes the configuration reusable across regions and across students.

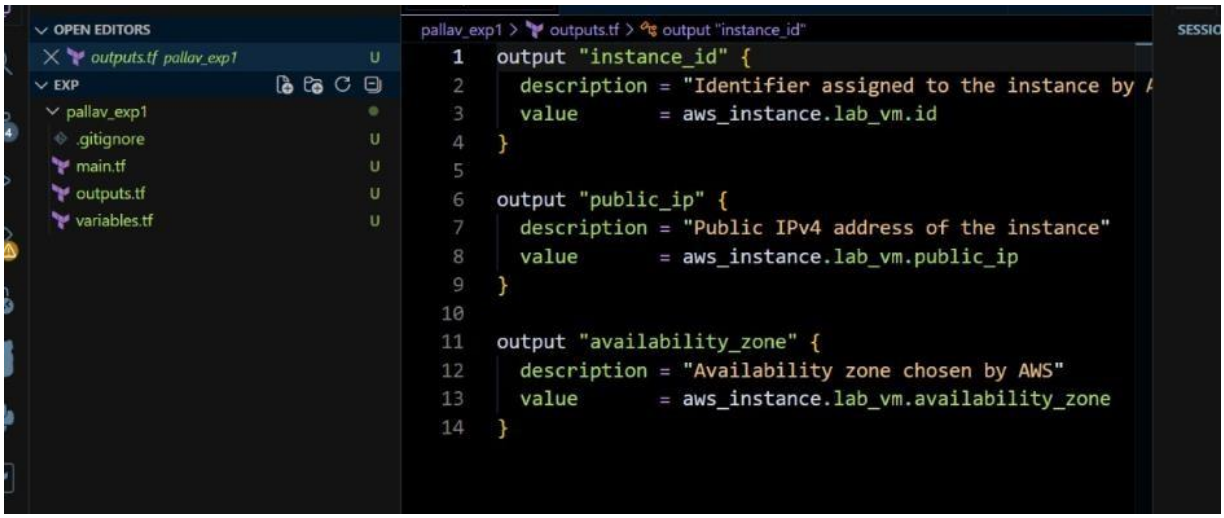

EXPLORER
OPEN EDITORS
variables.tf pallav_exp1
EXP
pallav_exp1
.gitignore
main.tf
outputs.tf
variables.tf
OUTLINE
TIMELINE
variables.tf
pallav_exp1 > variables.tf > variable "region"
variable "region" {
description = "AWS region in which the instance is created"
type = string
default = "ap-south-1"
}
variable "ami_id" {
description = "AMI identifier of Ubuntu 22.04 LTS in the chosen region"
type = string
default = "ami-0f58b397bc5c1f2e8"
}
variable "instance_type" {
description = "EC2 instance size; must remain within the free tier"
type = string
default = "t3.micro"
validation {
condition = contains(["t3.micro", "t3.micro"], var.instance_type)
error_message = "Only free-tier instance types are permitted in this lab."
}
}
variable "vm_name" {
description = "Value of the Name tag applied to the instance"
type = string
default = "lab-vm-01"

Step 6 — Declare the virtual machine resource

Add the aws_instance resource to main.tf. The resource block has two labels: the resource type, which is fixed by the provider, and a local name used to reference the resource elsewhere in the configuration. Tags are not optional in practice — untagged resources cannot be attributed to an owner and are the main cause of runaway cloud bills in student accounts.

Step 7 — Declare outputs

Create outputs.tf and export the instance identifier and the public IP address. Outputs are printed at the end of a successful apply and can be queried later with terraform output, which avoids having to open the cloud console to find the address.



Step 8 — Initialise the working directory

Run terraform init. The command downloads the AWS provider plugin, creates the .terraform directory and writes .terraform.lock.hcl. Commit the lock file to Git; it pins the provider version and its checksum for every future run.

Output of terraform init

```
PS C:\Users\palla\Desktop\exp\pallav_exp1> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\palla\Desktop\exp\pallav_exp1> terraform fmt
PS C:\Users\palla\Desktop\exp\pallav_exp1> terraform validate
Success! The configuration is valid.
```

Step 9 — Format and validate the configuration

Run terraform fmt to normalise indentation and alignment, then terraform validate to check that the configuration is syntactically correct and internally consistent. Validation does not contact AWS, so it runs in under a second and should be used continuously while editing.

Formatting and validating

```
PS C:\Users\palla\Desktop\exp\pallav_exp1> terraform fmt
PS C:\Users\palla\Desktop\exp\pallav_exp1> terraform validate
Success! The configuration is valid.
```

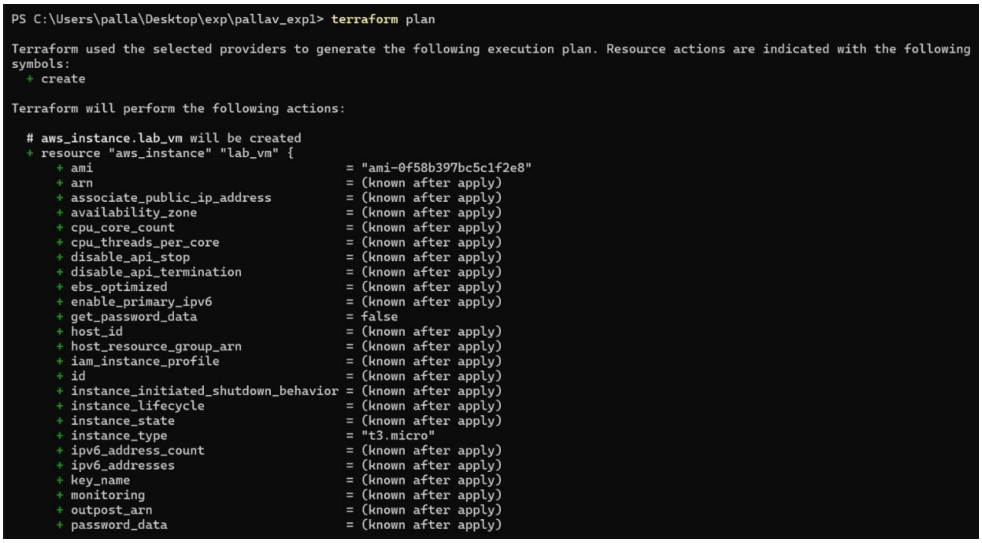
Step 10 — Generate and review the execution plan

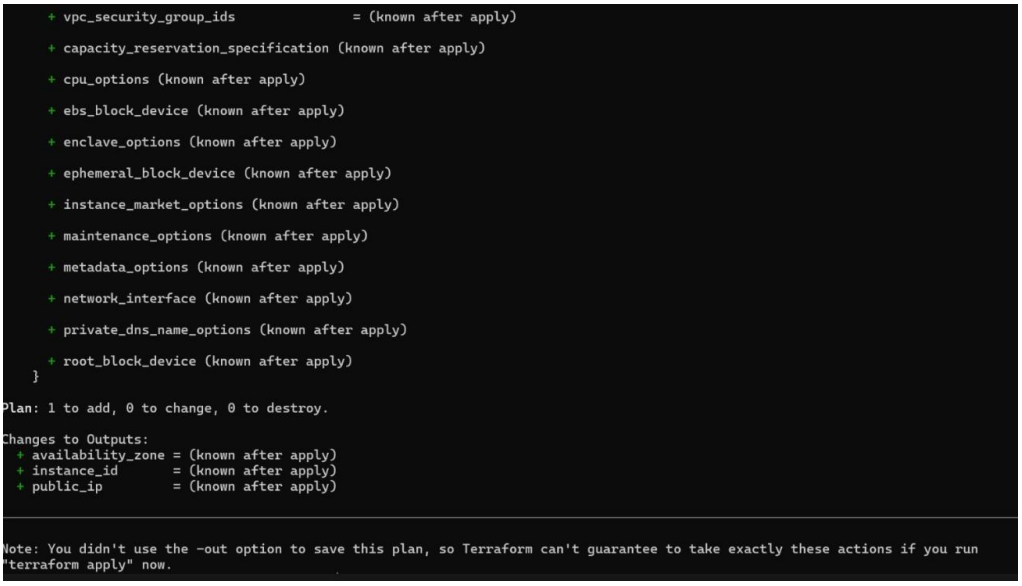
Run terraform plan. Read every line of the output. The symbol + indicates creation, ~ indicates in-place modification, and -/+ indicates that the resource must be destroyed and recreated. Values shown as (known after apply) are assigned by AWS and cannot be predicted. The summary line at the end must read exactly "Plan: 1 to add, 0 to change, 0 to destroy".

```
PS C:\Users\palla\Desktop\exp\pallav_exp1> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.lab_vm will be created
  + resource "aws_instance" "lab_vm" {
      + ami                                  = "ami-0f58b397bc5c1f2e8"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
```

Step 11 — Apply the configuration

Run terraform apply. Terraform recomputes the plan, displays it and waits for confirmation. Type yes at the prompt. Creation of an EC2 instance normally completes in thirty to sixty seconds. Record the instance ID and public IP printed in the outputs section.





```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

availability_zone = "ap-south-1b"
instance_id = "i-03d3c521e4517eb07"
public_ip = "43.204.108.214"
PS C:\Users\palla\Desktop\exp\pallav_exp1> terraform output
availability_zone = "ap-south-1b"
instance_id = "i-03d3c521e4517eb07"
public_ip = "43.204.108.214"
PS C:\Users\palla\Desktop\exp\pallav_exp1>
```

Step 12 — Verify the running instance

Confirm the result independently of Terraform, both through the AWS CLI and through the EC2 section of the AWS Management Console. Verification through a second channel is essential; a successful apply proves that the API call succeeded, not that the machine is in the expected state.

Independent verification through the AWS CLI

```
PS C:\Users\palla\Desktop\exp\pallav_exp1> aws ec2 describe-instances --instance-ids (terraform output -raw instance_id) --query "Res
ervations[].Instances[].{State:State.Name,Type:InstanceType,IP:PublicIpAddress,AZ:Placement.AvailabilityZone}" --output table
-------------------------------------------------------------
|                    DescribeInstances                      |
+--------------+-----------------+-----------+-------------+
|      AZ      |       IP        |  State    |   Type      |
+--------------+-----------------+-----------+-------------+
|  ap-south-1b |  43.204.108.214 |  running  |  t3.micro   |
+--------------+-----------------+-----------+-------------+

PS C:\Users\palla\Desktop\exp\pallav_exp1>
```

Step 13 — Commit the configuration

Stage and commit the .tf files and the lock file. Verify with git status that no state file or credential file has been staged.

Committing the configuration

```
$ git add main.tf variables.tf outputs.tf .gitignore .terraform.lock.hcl
$ git commit -m 'Experiment I: provision a single EC2 instance with
Terraform'
```

Step 14 — Destroy the infrastructure

Run terraform destroy and confirm with yes. This step is mandatory. A t2.micro instance left running consumes the Free Tier allowance and will be billed once the allowance is exhausted. Verify in the console that the instance state has changed to terminated.

```
PS C:\Users\palla\Desktop\exp\pallav_exp1> terraform apply
aws_instance.lab_vm: Refreshing state... [id=i-03d3c521e4517eb07]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

availability_zone = "ap-south-1b"
instance_id = "i-03d3c521e4517eb07"
public_ip = "43.204.108.214"
PS C:\Users\palla\Desktop\exp\pallav_exp1>
```

Program and Configuration Listings

Listing 1.1 — main.tf

```
terraform         {
  required_version
  =    ">=    1.6.0"
  required_provider
  s { aws = { source
  =   "hashicorp/aw
  s" version =
      "~> 5.0"
    }
  }
}

 tags
= {        =
  ExperimentNa
var.vm_na = "01"
 }ManagedBy =
}
```

Listing 1.2 — variables.tf

```
variable "region" {
  description = "AWS region
  in which the instance is
  created" type = string
  default        = "ap-south-
  1"
}
 variable "ami_id" { description = "AMI
identifier of Ubuntu
  22.04 LTS in the chosen region" type
              = string
  default        = "ami-
0f58b397bc5c1f2e8" }
 variable "instance_type" {
description = "EC2 instance size;
  must remain within the free tier"
  type        = string
  default     = "t2.micro"
```

Listing 1.3 — outputs.tf

```
output "instance_id" {
  description = "Identifier
  assigned to the instance
  by AWS" value =
  aws_instance.lab_vm.id
}
 output "public_ip"
{ description =
"Public IPv4
```

Expected Output

Output 1.1 — terraform plan (abridged)

```
$ terraform plan


Terraform used the selected providers
to generate the following execution
plan. Resource actions are indicated
with the following symbols: + create

Terraform will
  perform the
  following
  actions: #
```

Output 1.2 — terraform apply and terraform destroy

```
$ terraform apply
  ... (plan repeated) ...
Do you want to perform these
actions? Terraform will perform the
actions described above. Only 'yes'
will be accepted to approve. E nte r
a val
```

Observation Table

Stage	Command	Observed result
Initialisation	terraform init	AWS provider v5.62.0 installed; lock file created
Validation	terraform validate	Success! The configuration is valid.
Planning	terraform plan	Plan: 1 to add, 0 to change, 0 to destroy
Provisioning	terraform apply	1 resource added in 32 seconds
Verification	aws ec2 describe-instances	State reported as running, type t2.micro
Idempotency check	terraform apply (second run)	No changes. Infrastructure matches the configuration
Teardown	terraform destroy	1 resource destroyed; state file emptied

Result

A single Linux virtual machine was successfully provisioned on AWS from a declarative Terraform configuration, verified through both the CLI and the management console, and destroyed cleanly. The concepts of declarative infrastructure, idempotency and state management were demonstrated in practice.

Precautions

- Never commit terraform.tfstate, terraform.tfvars or any credential file to a Git repository. Verify with git status before every commit.
- Always read the full output of terraform plan before typing yes. The plan is the only safety gate between the configuration and the cloud account.
- Use only free-tier instance types. The validation block in variables.tf enforces this, but the check can be bypassed by editing the file.

Evaluation Rubric

Signature of the faculty in-charge: _____

Name: Pallavi Bharti
Sap id: 500119584
Roll: R2142231193
Batch: 2 DevOps.

① Difference between Terraform plan and apply.

Terraform plan	Terraform Apply
• Shows what Terraform will do.	• Actually performs the changes.
• Doesnot create or modify infrastructure.	• Creates, modifies, or destroys infrastructure.
• Used for reviewing changes before execution.	• Used for provisioning the planned infrastructure.
• Acts as a safety check	• Executes the approved changes.

② What is stored in state file and why is it sensitive?

→ Terraform.tfstate stores the mapping between the Terraform configuration and the real infrastructure created in AWS. It contains resource information and cloud-assigned identifiers such as an EC2 instance ID.
It is sensitive because the state files may contain resource details and sensitive values.

State file: Terraform's record of the infrastructure it manages.

(3.) Define idempotency and give an example from this experiment.

→ Idempotency means performing the same operation repeatedly produces the same result without creating duplicate resources.
In this experiment, running terraform apply again after successful creation showed "No changes", so no new EC2 instance was created.

(4.) Configuration drift and How terraform detects?

→ Configuration drift occurs when the actual AWS infrastructure differs from the terraform config.

It detects by comparing the current infrastructure with the desired configuration during terraform plan.

(5.) Why is a version constraint placed on the provider?

→ A provider version constraint ensures compatibility and helps maintain a stable and reproducible Terraform configuration.

(6.) What is the purpose of the .terraform.lock.hcl file?

→ It records the selected provider version and checksums, ensuring the same compatible

provides it used in future runs.

(7.) Why is the declarative model preferred over imperative shell script?

→ The declarative model describes what the desired infrastructure should be, while Terraform determines how to achieve it.

It provides → consistency
↳ Automation
↳ Idempotency
↳ change detection

Therefore, Terraform focuses on what the infrastructure should look like, while an imperative script mainly specifies how to create it step by step.